

Gleichungssysteme und Matrixzerlegungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Algorithmen:** Komplexität, Algorithmustypen (direkte Methoden)
- **Fehleranalyse:** Stabilität, Kondition von Problemen
- Matrixnormen, Orthogonalmatrizen
- **Lineare Algebra I**
 - Lösung linearer Gleichungssysteme (grundlegende Methoden)
 - Dreiecksmatrizen, Symmetrische Matrizen, Positiv definite Matrizen
 - Matrixinverse (wir lernen, dass wir sie *NICHT* berechnen...!)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\mathbf{U} \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix mit $u_{ii} \neq 0$. Mit welchem Aufwand lässt sich $\mathbf{U}\mathbf{x} = \mathbf{c}$ lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. Gar nicht — dazu muss man erst $\mathbf{U}^{-1}$ berechnen.

Frage 2: Was bedeutet “ $\mathbf{A} \in \mathbb{R}^{n \times n}$ ist *positiv definit*”?

1. Alle Einträge von $\mathbf{A}$ sind positiv.
2. $\det(\mathbf{A}) > 0$.
3. $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ für alle $\mathbf{x} \neq \mathbf{0}$.
4. Alle Eigenwerte von $\mathbf{A}$ sind verschieden.

Numerische lineare Algebra

- Wir kümmern uns nun um numerische Methoden für die lineare Algebra:
 - Matrizenrechnung
 - Lineare Gleichungssysteme
 - Kleinste-Quadrate-Probleme
 - Eigenwerte
 - ...
- Dabei achten wir insbesondere auf **Kondition** und **Komplexität**.

Matrizenrechnung

Matrizenrechnung

- Die algorithmische Berechnung folgt für gewöhnlich den direkten mathematischen Formeln

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a_{11} + b_{11} & \cdots & a_{1m} + b_{1m} \\ \vdots & & \vdots \\ a_{n1} + b_{n1} & \cdots & a_{nm} + b_{nm} \end{pmatrix},$$

$$\mathbf{AB} = \begin{pmatrix} \sum_{i=1}^m a_{1i}b_{i1} & \cdots & \sum_{i=1}^m a_{1i}b_{ik} \\ \vdots & & \vdots \\ \sum_{i=1}^m a_{ni}b_{i1} & \cdots & \sum_{i=1}^m a_{ni}b_{ik} \end{pmatrix}.$$

Quiz

Was ist die algorithmische Komplexität für die Addition von $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times m}$?

1. $O(nm)$
2. $O(n + m)$
3. $O(n^2m^2)$

Quiz

Was ist die algorithmische Komplexität für die Multiplikation von $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\mathbf{B} \in \mathbb{R}^{m \times k}$?

1. $O(nk)$
2. $O(nmk)$
3. $O(nm + k)$

Matrixmultiplikation

- Für die Multiplikation von zwei $(n \times n)$ Matrizen gibt es Algorithmen mit besserer Komplexität.
- Für $A, B \in \mathbb{R}^{n \times n}$:
 - Strassen (1969): $\approx O(n^{2.8})$
 - Alman et al. (2024): $\approx O(n^{2.372})$ (bester bekannter Exponent)



Quelle: Titelbild *Nature* **610** (6. Oktober 2022) zu AlphaTensor (Fawzi et al., 2022).

- Die Konstante, die hinter $O(\cdot)$ versteckt wird, ist aber meist so groß, dass die Algorithmen kaum verwendet werden.
- Wir rechnen der Einfachheit halber immer mit $O(nmk)$.

Lineare Gleichungssysteme

Lineare Gleichungssysteme

- Wir beschäftigen uns nun mit der Lösung eines LGS:

$$\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

- Das ist nur sinnvoll, wenn es (genau) eine Lösung gibt. Also nehmen wir an, dass $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ invertierbar ist.
- Die Lösung kann als $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$ geschrieben werden.
- Das numerische Invertieren einer Matrix ist aber
 - relativ instabil und
 - oft langsamer als direktes Lösen des LGS.

Deshalb:

Invertiere niemals eine Matrix!

Demo:

- LGS mit schlecht konditionierter Matrix mit $\kappa_2(\boldsymbol{A}) \approx 10^{14}$
- wahre Lösung $\boldsymbol{x} = (1, \dots, 1)^\top$:

```
1 n <- 11 # Hilbert-Matrix:
2 A <- outer(1:n, 1:n,
3           function(i, j) 1 / (i + j - 1))
4 x <- rep(1, n)
5 b <- A %*% x
6
7 range((solve(A) %*% b) - x) # Fehler über Inverse
8 range(solve(A, b) - x)      # Fehler direkte Lösung
```

```
[1] -0.195  0.288
[1] -0.000260  0.000332
```

$\implies$ maximaler Fehler über $\boldsymbol{A}^{-1}$: ≈ 0.3 ;
bei direktem Lösen: $\approx 3 \cdot 10^{-4}$
— ca. **1000× genauer** (und schneller).

Lineare Gleichungssysteme

Wir erinnern uns an das Gauss'sche Eliminationsverfahren:

1. Bringe $(\mathbf{A} \mid \mathbf{b})$ in Zeilenstufenform $(\mathbf{U} \mid \mathbf{c})$ mit oberer Dreiecksmatrix $\mathbf{U}$.
2. Löse das System

$$\begin{array}{ccccccccc} u_{11}x_1 & + & u_{12}x_2 & + & \dots & + & u_{1n}x_n & = & c_1 \\ & & u_{22}x_2 & + & \dots & + & u_{2n}x_n & = & c_2 \\ & & & & \ddots & & \vdots & & \vdots \\ & & & & & & u_{nn}x_n & = & c_n \end{array}$$

durch Rückwärtssubstitution

$$x_n = c_n/u_{nn}, \quad x_i = \left(c_i - \sum_{j=i+1}^n u_{ij}x_j \right) / u_{ii} \quad (i \leq n-1).$$

Quiz

Was ist die algorithmische Komplexität der Rückwärtssubstitution?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Warum Matrixzerlegungen?

Grundidee: “Teile und herrsche” für Matrizen

Analogie: Primfaktorzerlegung von Zahlen

- $84 = 2^2 \cdot 3 \cdot 7$
- Vereinfacht Rechnung: $84 \cdot 126 = (2^2 \cdot 3 \cdot 7) \cdot (2 \cdot 3^2 \cdot 7) = 2^3 \cdot 3^3 \cdot 7^2$

Für Matrizen: $A = BC$ mit “einfachen” B, C

- **Dreiecksmatrizen:** Lösen in $O(n^2)$ statt $O(n^3)$
- **Orthogonale Matrizen:** Numerisch stabil, einfach zu invertieren
- **Diagonalmatrizen:** Triviale Operationen

Vorteile von Zerlegungen:

1. Effiziente Lösung mehrerer LGS mit gleicher Matrix
2. Numerische Stabilität verbessern
3. Struktur und Eigenschaften von A erkennen
4. Grundlage für viele weitere Algorithmen (Spektralzerlegung & SVD, ...)

Die LU-Zerlegung

Intuition: LU-Zerlegung

Idee: Die Gauss-Elimination zerlegt A in:

- L (Lower): speichert alle Eliminationsschritte
- U (Upper): die resultierende obere Dreiecksmatrix

Vorteil: Sobald wir $A = LU$ einmal berechnet haben, können wir $Ax = b_i$ für jedes b_i effizient lösen:

$$Ax = b \implies LUx = b \implies \begin{array}{l} 1. \text{ Löse } Ly = b \\ 2. \text{ Löse } Ux = y \end{array}$$

Die LU-Zerlegung

- Im ersten Schritt des Eliminationsverfahrens bringen wir $\mathbf{A} \in \mathbb{R}^{n \times n}$ in Zeilenstufenform.
- Dazu gehen wir von links nach rechts durch die Spalten und entfernen alle Einträge unter der Diagonalen.
- Ist das aktuelle System $(\mathbf{M} \mid \mathbf{r})$, erreichen wir das für die k -te Spalte durch die Multiplikation einer Matrix $\mathbf{L}_k$ von links, wobei

$$\mathbf{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{pmatrix}$$

mit $l_{ik} = m_{ik}/m_{kk}$

Die LU-Zerlegung

- Insgesamt ist also $(U \mid c)$ gegeben durch $U = L_{n-1} \cdots L_1 A$ und $c = L_{n-1} \cdots L_1 b$.
- Es gilt dann $A = LU$, wobei $L = L_1^{-1} \cdots L_{n-1}^{-1}$.

Die Inverse zu L_k macht deren Operation rückgängig (Vorzeichen kippen):

Jedes L_k^{-1} ist eine untere Dreiecksmatrix, also auch $L = L_1^{-1} \cdots L_{n-1}^{-1}$ – genauer:

$$L_k^{-1} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & & 1 \end{pmatrix}$$

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & \ddots & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & & \ddots & \ddots & \\ l_{n1} & \cdots & \cdots & l_{n(n-1)} & 1 \end{pmatrix}$$

$\implies L$ sammelt einfach alle Eliminationskoeffizienten l_{ik} ein.

Beispiel: LU-Zerlegung für 3×3 -Matrix

$$\text{Sei } A = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}.$$

Farben: zu eliminieren, Pivot, Eliminationskoeffizienten

Schritt 1: Eliminiere Spalte 1 mit L_1 :

$$l_{21} = 4/2 = 2, \quad l_{31} = -2/2 = -1$$

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -(-1) & 0 & 1 \end{pmatrix},$$

$$L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 8 & 1 \end{pmatrix}$$

Schritt 2: Eliminiere Spalte 2 mit L_2 :

$$l_{32} = 8/(-8) = -1$$

$$L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -(-1) & 1 \end{pmatrix},$$

$$L_2 L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} = U$$

Ergebnis: $A = LU$ mit

$$L = L_1^{-1} L_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix}$$

Die LU-Zerlegung zur Lösung von LGS

Theorem: LU-Zerlegung

Jede invertierbare Matrix A hat – ggf. erst nach Zeilenvertauschungen $A \rightarrow PA$ mit einer Permutationsmatrix P ($\rightarrow$ *Pivotierung*, s.u.) – eine Darstellung

$$A = LU \quad \text{bzw.} \quad PA = LU$$

mit

- einer unteren Dreiecksmatrix L ,
 - einer oberen Dreiecksmatrix U .
-
- Eine Zerlegung ohne Pivotierung (ohne Zeilenvertauschungen, also mit $P = I$) erhalten wir genau dann, wenn bei der Gauß-Elimination kein Pivotelement null wird. Wird ein Pivot null, müssen wir vorher Zeilen vertauschen.
 - Da L und U Dreiecksmatrizen sind, können wir das LGS $Ax = b$ bzw. $LUx = b$ jetzt leicht lösen:
 1. Löse $Ly = b$ (bzw. $Ly = Pb$) durch Vorwärtssubstitution.
 2. Löse $Ux = y$ durch Rückwärtssubstitution.
 - **Beachte:** Die Zerlegung $A = LU$ ist für alle b gleich. Wir müssen sie also nur einmal ausführen.

Beispiel: Lösen mit LU-Zerlegung (Fortsetzung)

Mit L, U von vorher und $b = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix}$.

Schritt 1: Vorwärtssubstitution für $Ly = b$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix} \implies \begin{array}{lcl} y_1 & = & 5 \\ y_2 & = & -2 - 2 \cdot 5 = -12 \\ y_3 & = & 9 - (-1) \cdot 5 - (-1) \cdot (-12) = 9 + 5 - 12 = 2 \end{array}$$

Schritt 2: Rückwärtssubstitution für $Ux = y$:

$$\begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 2 \end{pmatrix} \implies \begin{array}{lcl} x_3 & = & 2/3 \\ x_2 & = & (-12 - 2 \cdot (2/3))/(-8) = 5/3 \\ x_1 & = & (5 - 1 \cdot (5/3) - (-1) \cdot (2/3))/2 = 2 \end{array}$$

Pivotierte LU-Zerlegung

- Die Methode scheitert, falls im k -ten Schritt $m_{kk} = 0$.
- Das lässt sich vermeiden, indem wir Zeilen von $\mathbf{A}$ tauschen: → **Pivotierung**
 - löst $\mathbf{PA} = \mathbf{LU}$ mit Permutationsmatrix $\mathbf{P}$
- *pivoting* verbessert Stabilität auch schon für $m_{kk} \approx 0$.

Quiz

Im k -ten Schritt zum Finden von $\mathbf{L}$ und $\mathbf{U}$ multiplizieren wir

$$\mathbf{M} = \begin{pmatrix} * & \dots & * & \dots & * \\ & \ddots & \vdots & \ddots & \vdots \\ & & * & \dots & * \\ & & \vdots & \ddots & \vdots \\ & & * & \dots & * \end{pmatrix} \text{ mit } \mathbf{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & -l_{(k+1)k} & & 1 & \\ & \vdots & & & \ddots \\ & -l_{nk} & & & & 1 \end{pmatrix}.$$

Was ist die Komplexität dieses Schritts?

1. $O(n^2)$
2. $O(n^3)$
3. $O((n - k)^2)$
4. $O(k^3)$

Komplexität

- Die Berechnung der LU-Zerlegung hat also die Komplexität

$$\sum_{k=1}^{n-1} O((n-k)^2) = \sum_{k=1}^{n-1} O(n^2) = O(n^3).$$

- Das müssen wir nur einmal ausführen und können damit mehrere LGS $\mathbf{Ax} = \mathbf{b}_j, j = 1, \dots, J$, lösen.
- Für Vorwärts- und Rückwärtssubstitution brauchen wir je $O(n^2)$.
- Insgesamt ist die Laufzeit also $O(n^3 + Jn^2)$.
- Die Laufzeit kann verbessert werden, wenn $\mathbf{A}$ besondere Eigenschaften erfüllt, z.B.:
 - symmetrisch: $\mathbf{A} = \mathbf{A}^\top$,
 - positiv definit: $\mathbf{x}^\top \mathbf{Ax} > 0$ für alle $\mathbf{x} \neq \mathbf{0}$,
 - Bandeigenschaft: $a_{ij} = 0$ für alle $|i - j| > \beta$,
 - dünn besetzt (engl.: *sparse*): $a_{ij} = 0$ für die allermeisten (i, j) .
- Für all das gibt es hervorragende Implementierungen. Man schreibt diese Algorithmen selten selbst.

Cholesky-Zerlegung

SPD-Matrizen

Ein Spezialfall ist in der Statistik besonders wichtig:

SPD-Matrizen sind

- **s**ymmetrisch und
- **p**ositiv **d**efinit.

Def.: SPD-Matrix

Eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ heißt **SPD**, wenn

- $\mathbf{A} = \mathbf{A}^\top$ (*symmetrisch*) und
 - $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0 \ \forall \ \mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ (*positiv-definit*).
-
- Symmetrisches $\mathbf{A}$ ist **positiv-semidefinit** (PSD) $\iff \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \ \forall \ \mathbf{x} \in \mathbb{R}^n$

SPD- und PSD-Matrizen in Statistik und ML

1. Kovarianzmatrizen: $\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$

sind immer PSD, und SPD, wenn $\mathbf{x}$ nicht auf einer Untermenge von $\mathbb{R}^p$ mit Dimension $< p$ liegt.

- Multivariate Normalverteilungen $\sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$, Hauptkomponentenanalyse (PCA)

2. Gram-Matrizen: $\mathbf{K} = \mathbf{X}^\top \mathbf{X}$

- Normalgleichungen $\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$ für **Kleinste-Quadrate**-Probleme; Kernel-Matrizen (SVMs, Gauss-Prozesse, ...)

3. Hesse-Matrizen: $\mathbf{H} = \left[\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) \right]_{i,j}$

- Fisher-Information; **Konvexität** & **Newton-Verfahren** in der Optimierung

Warum so wichtig?

SPD-Matrizen garantieren:

- Existenz und — bei positiver Diagonale von $\mathbf{L}$ — Eindeutigkeit der Cholesky-Zerlegung
- Numerische Stabilität der Cholesky-Zerlegung (auch ohne Pivotierung)
- Alle Eigenwerte sind (reell und) positiv $\implies$ voller Rang

Cholesky-Zerlegung

- Für SPD-Matrizen kann man $\mathbf{L}$ und $\mathbf{U}$ so wählen, dass $\mathbf{U} = \mathbf{L}^\top$.

Theorem: Cholesky-Zerlegung

Jede SPD-Matrix $\mathbf{A}$ kann in $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ zerlegt werden, wobei $\mathbf{L}$ eine untere Dreiecksmatrix mit positiver Diagonale ($l_{ii} > 0$) ist. Diese Zerlegung ist eindeutig.

- Der Rechen- und Speicheraufwand ist hier etwa halb so groß wie für die volle LU-Zerlegung.

(Beweis per Induktion $\rightarrow$ Anhang)

Beispiel: Cholesky-Zerlegung

Berechne Cholesky-Zerlegung von $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Ansatz: $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ mit $\mathbf{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$

Ausmultiplizieren:

$$\mathbf{L}\mathbf{L}^\top = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Koeffizientenvergleich:

$$\begin{aligned} l_{11}^2 &= 4 & \implies & l_{11} = 2 \\ l_{11}l_{21} &= 2 & \implies & l_{21} = 2/2 = 1 \\ l_{21}^2 + l_{22}^2 &= 3 & \implies & l_{22} = \sqrt{3-1} = \sqrt{2} \end{aligned}$$

Ergebnis:

$$\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \implies \mathbf{L}\mathbf{L}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Cholesky-Zerlegung

Beispielanwendung:

- Sei $\mathbf{x} \in \mathbb{R}^d$ ein Zufallsvektor mit $\mathbb{E}[\mathbf{x}] = \mathbf{0}$ und $\text{Var}(\mathbf{x}) = \mathbf{I}_d = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$.
- Sei $\mathbf{\Sigma} \in \mathbb{R}^{d \times d}$ eine SPD Matrix und $\mathbf{\Sigma} = \mathbf{L}\mathbf{L}^\top$.
- Dann hat $\mathbf{y} = \mathbf{L}\mathbf{x}$ die Kovarianz

$$\text{Var}(\mathbf{y}) = \mathbb{E}[\mathbf{y}\mathbf{y}^\top] = \mathbb{E}[\mathbf{L}\mathbf{x}\mathbf{x}^\top\mathbf{L}^\top] = \mathbf{L}\mathbb{E}[\mathbf{x}\mathbf{x}^\top]\mathbf{L}^\top = \mathbf{L}\mathbf{L}^\top = \mathbf{\Sigma}.$$

(das **mittlere Stück** fällt weg: $\mathbb{E}[\mathbf{x}\mathbf{x}^\top] = \mathbf{I}_d$ nach Voraussetzung)

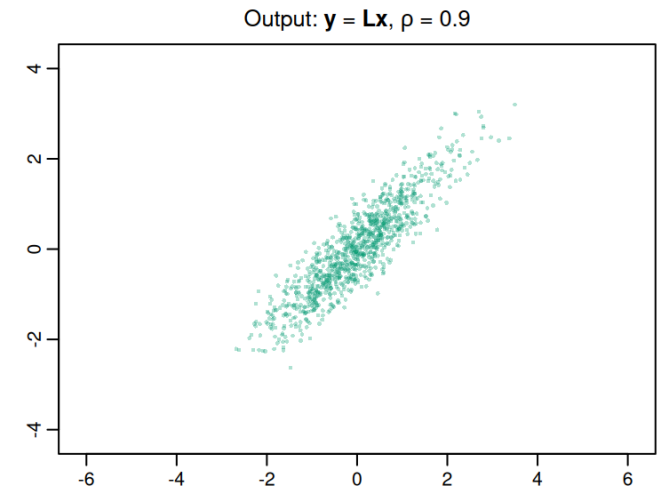
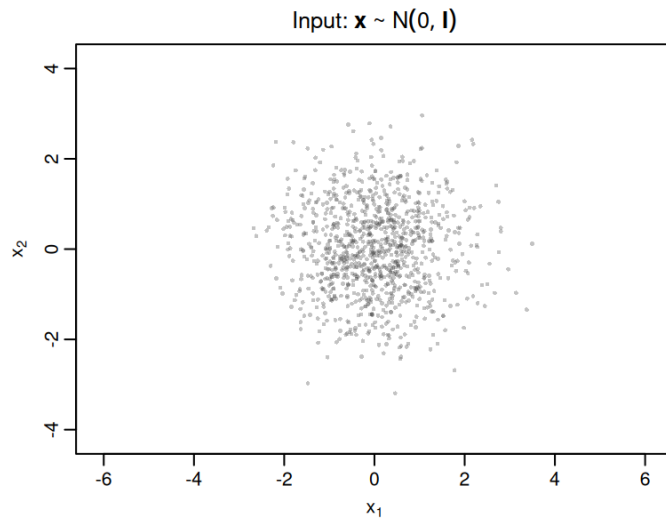
- Dies wird z.B. verwendet, um aus $\mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$ zu simulieren:
für $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ ist $\boldsymbol{\mu} + \mathbf{L}\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$.

Demo: Aus $\mathcal{N}(\mathbf{0}, \Sigma)$ simulieren

```
1 set.seed(7)
2 Sigma <- matrix(c(1, 0.9,
3 0.9, 1), 2, 2)
4 # !! chol() liefert die obere Dreiecksmatrix R mit Sigma = R'R !!
5 L <- t(chol(Sigma))
6 x <- matrix(rnorm(2 * 1000), nrow = 2)
7 y <- L %*% x
8 round(cov(t(y)), 2)
```

```
      [,1] [,2]
[1,] 1.00 0.89
[2,] 0.89 0.99
```

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 0.9 & \sqrt{1 - 0.9^2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.9 & 0.436 \end{pmatrix}, \quad \Sigma = \mathbf{L}\mathbf{L}^\top.$$



$\implies \mathbf{L}$ verformt Gitter und Punktwolke auf dieselbe Weise; die empirische Kovarianz von $\mathbf{y}$ reproduziert Σ .

Vergleich: LU und Cholesky

LU-Zerlegung

- **Ohne Pivotierung instabil** bei kleinen Pivotelementen
 - Division durch sehr kleine Zahlen führt zu numerischer Instabilität
- **Mit partieller Pivotierung** praktisch stabil (Higham, 2020)
- **Kondition:** Fehler $\approx \kappa(\mathbf{A}) \cdot \epsilon_{\text{Maschine}}$

Cholesky-Zerlegung

- **Natürlich stabil** ohne Pivotierung für *SPD-Matrizen*
- **Effizienz:** $\approx 2\times$ schneller als LU (halber Aufwand)
- **Robustheit:** In exakter Arithmetik bricht das Verfahren für **jede** nicht positiv definite Matrix ab (Wurzel aus einer nicht-positiven Zahl); bei echten SPD-Matrizen passiert das nur durch Rundungsfehler und nur bei **extrem** schlechter Kondition
 - Der Abbruch ist ein nützlicher *Diagnosehinweis*: $\mathbf{A}$ ist nicht SPD oder numerisch grenzwertig
- **Pivotierte Variante:** Anwendung auch für *semi*-positiv-definite Matrizen! (Details $\rightarrow$ Anhang)

Fazit

- *LU*: Universell einsetzbar, benötigt Pivotierung für Stabilität
- *Cholesky*: optimal für *SPD-Matrizen* (Statistik!), numerisch überlegen

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- **Invertiere niemals eine Matrix!**
- *LU*- und *Cholesky*-Zerlegungen

Nächste Vorlesung

- Singulärwertzerlegung (*SVD*)

Self-Check

Frage 1: Sie müssen viele LGS $\mathbf{Ax} = \mathbf{b}_j$ mit derselben SPD-Matrix $\mathbf{A}$ lösen. Bestes Vorgehen?

1. Für jedes $\mathbf{b}_j$ die Gauss-Elimination komplett neu durchführen.
2. Einmal $\mathbf{A}^{-1}$ berechnen, dann jeweils $\mathbf{A}^{-1}\mathbf{b}_j$ multiplizieren.
3. Einmal Cholesky $\mathbf{A} = \mathbf{LL}^\top$, dann je zweimal Substitution.
4. Einmal pivotierte LU, da sie für SPD-Matrizen stabiler als Cholesky ist.

Frage 2: Wann braucht die LU-Zerlegung Pivotierung?

1. Nur wenn $\mathbf{A}$ singulär ist.
2. Wenn ein Pivotelement m_{kk} null oder nahe null ist.
3. Nie — sie verbessert nur die Laufzeit.
4. Nur für symmetrische Matrizen.

Frage 3: $\mathbf{A} = \mathbf{LU}$ liegt bereits vor. Kosten, um *ein* LGS $\mathbf{Ax} = \mathbf{b}$ zu lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. $O(n^2 + n^3)$

Anhang

Cholesky-Zerlegung (Induktionsbeweis)

- $n = 1$: Weil $A \in \mathbb{R}$ SPD ist, gilt $A > 0$. Dann ist $A = LL^\top$ mit $L = \sqrt{A}$.

- $n > 1$:

- Sei $A = \begin{pmatrix} a & \mathbf{c}^\top \\ \mathbf{c} & B \end{pmatrix}$ und die Hypothese gelte für $n - 1$.

- Setze $\mathbf{l} = \frac{\mathbf{c}}{\sqrt{a}}$. (A SPD $\implies a > 0$, da $a = \mathbf{e}_1^\top A \mathbf{e}_1$.)

- Außerdem ist für jedes $\mathbf{x} \in \mathbb{R}^{n-1} \setminus \{\mathbf{0}\}$:

$$\mathbf{x}^\top (B - \mathbf{u}^\top) \mathbf{x} = \begin{pmatrix} -\frac{\mathbf{c}^\top \mathbf{x}}{a} \\ \mathbf{x} \end{pmatrix}^\top A \begin{pmatrix} -\frac{\mathbf{c}^\top \mathbf{x}}{a} \\ \mathbf{x} \end{pmatrix} > 0$$

also ist $(B - \mathbf{u}^\top)$ SPD und $(B - \mathbf{u}^\top) = \tilde{L}\tilde{L}^\top$ (Induktionshypothese).

- Wähle nun

$$L = \begin{pmatrix} \sqrt{a} & \mathbf{0}^\top \\ \mathbf{l} & \tilde{L} \end{pmatrix} \implies LL^\top = \begin{pmatrix} a & \mathbf{c}^\top \\ \mathbf{c} & \mathbf{u}^\top + \tilde{L}\tilde{L}^\top \end{pmatrix} = A \quad \checkmark$$

- Eindeutigkeit: $l_{11} = \sqrt{a}$ und $\mathbf{l} = \mathbf{c}/\sqrt{a}$ sind erzwungen, $\tilde{L}$ ist eindeutig nach Induktionsannahme.

↔ zurück: „Cholesky-Zerlegung“

Cholesky mit Pivotierung

- Notwendig falls $\mathbf{A}$ singular oder numerisch rangdefizient
 - für positiv **semi**-definite Matrizen ($\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ statt > 0):
- **Complete pivoting:** $\mathbf{P}^\top \mathbf{A} \mathbf{P} = \mathbf{L} \mathbf{L}^\top$
 - Permutationsmatrix $\mathbf{P}$ (zeilen- und spaltenvertauschend)
 - **Rang-aufdeckend:** Nullspalten in $\mathbf{L}$ zeigen Rangabfall von $\mathbf{A}$
- **Stabilität:** Auch für semidefinite Matrizen stabil (Higham, 1990)
- **Anwendungen** in ML/Statistik:
 - Kernel-Matrizen (Gauss-Prozesse, SVMs, ...)
 - Niedrig-rangige approximative Matrixzerlegung: $O(k^2 n)$ statt $O(n^3)$
 - Gram-Matrizen $\mathbf{X}^\top \mathbf{X}$ bei $p > n$ (mehr Features als Daten)

↔ zurück: „Vergleich: LU und Cholesky“